程序设计

讲义

2026 年秋季

毕朝日

复旦大学集成电路与微纳电子创新学院

zhaori_bi@fudan.edu.cn https://bzr2915.github.io/bzr2915/

目录

第六讲　函数：把操作组织起来 **4**

6.1　从重复输出到函数 4

6.1.1　给一段循环一个名字 4

6.1.2　声明、调用与定义 5

6.1.3　无返回值的函数 6

6.2　形参、实参与局部变量 7

6.2.1　一个函数，两项输入 7

6.2.2　实参复制给形参 8

6.2.3　每次调用都有独立的局部对象 9

6.3　返回值与返回路径 10

6.3.1　返回两个整数中的较小值 10

6.3.2　返回类型决定最终交回的值 11

6.3.3　提前返回与返回路径 12

6.4　递归：函数调用自身 13

6.4.1　进入下一层，再逐层返回 13

6.4.2　选读：一帧调用与递归的机械展平 14

6.4.3　终止条件与语句次序 17

6.4.4　阶乘：循环与递归的对照 17

6.4.5　斐波那契数列：重复子问题 19

6.4.6　十进制整数转成二进制 20

6.5　声明、头文件与多文件编译 21

6.5.1　头文件提供接口信息 21

6.5.2　把声明和定义放进不同文件 22

6.5.3　编译单元与链接 23

6.6　伪随机数：可重复的序列与变化的种子 24

6.6.1　`rand` 的返回值与范围 24

6.6.2　用 `srand` 设置起点 24

6.7　`inline` 是一项优化建议 25

6.8　指针：让函数找到调用者的对象 26

6.8.1 传值交换只修改形参 26
6.8.2 地址与指针对象 27
6.8.3 解引用：沿指针访问目标对象 28
6.8.4 用指针完成交换 29
6.8.5 重新理解 scanf 的地址参数 30
6.9 习题与解析 31
6.10 检查一次函数调用 35

第六讲　函数：把操作组织起来

要打印一份 40 字符宽的简单信头，第一行和最后一行各需要 40 个星号，中间一行显示公司名 `GIG INC.`。只在 `main` 中依次写输出语句，也能完成任务；但“连续打印若干个同一种字符”会出现多次。如果以后把星号改成等号，或者把 40 改成 60，就需要找到并修改每一处相似代码。

第五讲已经能用 `for` 循环打印 40 个星号。本讲先把这段循环命名为 `starbar`，需要时只写一次 `starbar()`。随后让函数接收字符和次数，使同一段代码既能打印星号，也能打印公司名前的空格。接下来需要建立两种连接：调用者把数据交给函数，函数把结果交回调用者。

6.1　从重复输出到函数

6.1.1　给一段循环一个名字

函数是一组具有名字、能够完成特定操作的代码。前几讲使用的 `printf`、`scanf`、`strlen` 和 `sqrt` 都是函数。程序员也可以定义自己的函数，把一段操作从 `main` 中分离出来。

下面的程序定义了 `starbar`。它不接受参数，也不返回计算结果；每次被调用，就打印 40 个星号和一个换行符。

```
#include <stdio.h>
#define WIDTH 40

void starbar(void);

int main(void) {
    starbar();
    printf("GIG INC.\n");
    starbar();
    return 0;
}

void starbar(void) {
    for (int cnt = 1; cnt <= WIDTH; cnt++) {
        putchar('*');
    }
    putchar('\n');
}
```

输出为：

```
****************************************
GIG INC.
****************************************
```

main 中的两处 starbar(); 不再重复循环的细节。若要把星号改成等号，只需修改 starbar 函数体中的 putchar。这就是模块化的一项直接收益：调用处说明“要做什么”，函数定义说明“具体怎样做”。

黑盒子是对这种分工的一种称呼。使用函数时，调用处关心输入、输出和前提，函数定义负责具体实现。参数允许什么范围、函数是否修改调用者的数据，都属于接口的一部分。

6.1.2 声明、调用与定义

starbar 这个名字在程序中出现于三个不同位置：

写法	作用
`void starbar(void);`	**函数声明**，也称函数原型。它告诉编译器函数名、返回类型以及参数个数和类型，末尾有分号。
`starbar();`	**函数调用**。程序转去执行该函数，函数结束后再回到调用处。
`void starbar(void) { ... }`	**函数定义**。函数头后接函数体，给出实际操作，函数头与花括号之间没有分号。

定义本身也提供声明。如果把完整的 `starbar` 定义放在 `main` 之前，就不需要再单独写前面的原型。本例把定义放在后面，因此先写原型，使编译器在分析 `main` 中的调用时已经知道接口。原型也可以写在调用所在的代码块内；本讲把它放在所有函数定义之外、源文件靠前的位置，后续函数都能使用同一份接口声明。

原型中的第一个 `void` 表示函数不返回值；圆括号中的 `void` 表示函数不接受参数。C11 中，`void starbar(void)` 与只写空圆括号的 `void starbar()` 含义不同：后者没有提供参数类型信息。本讲用 `(void)` 明确表示无参数。

6.1.3　无返回值的函数

`starbar` 的返回类型是 `void`，它通过输出字符产生作用，而不提供一个可赋给变量的结果。因此调用通常单独写成一条语句：

```
starbar();
```

不能写 `x = starbar();`，因为这个调用没有可放入 `x` 的值。

`void` 函数执行到函数体末尾的右花括号时自然结束。也可以在需要的位置写：

```
return;
```

这条语句立即结束本次调用，不携带返回值。`return;` 用于无返回值函数；有返回值函数则用 `return expression;` 交回结果。

调用函数的一方称为**主调函数**，被调用的一方称为**被调函数**。这两个名称描述一次调用中的关系，不是函数永久的身份：`main` 调用 `fun1` 时是主调函数；若 `fun1` 又调用 `fun2`，`fun1` 在下一次关系中就既是 `main` 的被调函数，又是 `fun2` 的主调函数。

本课程使用的 C 程序从 `main` 开始执行，再由 `main` 逐层调用其他函数。`main` 的返回值交给运行环境；到达 `main` 末尾等价于 `return 0;`。其他函数仍应写清自己的返回路径。

6.2 形参、实参与局部变量

6.2.1 一个函数，两项输入

星号函数只能处理一种字符和固定次数。为了打印公司名前的空格，可以把它推广为 show_n_char：第一个参数给出字符，第二个参数给出次数。函数自身不输出换行，因为同一行后面还可能要打印公司名。

下面的程序打印上下两条星号行，并在中间行左侧补空格。strlen(NAME) 给出公司名的字节数；本例的名字是 ASCII 字符，占 8 个字节，所以左侧空格数为 $(40-8)/2=16$。

```
#include <stdio.h>
#include <string.h>
#define NAME "GIG INC."
#define WIDTH 40
#define SPACE ' '

void show_n_char(char ch, int num);

int main(void) {
    int spaces;

    show_n_char('*', WIDTH);
    putchar('\n');
    spaces = (WIDTH - (int)strlen(NAME)) / 2;
    show_n_char(SPACE, spaces);
    printf("%s\n", NAME);
    show_n_char('*', WIDTH);
    putchar('\n');
    return 0;
}

void show_n_char(char ch, int num) {
    for (int cnt = 1; cnt <= num; cnt++) {
        putchar(ch);
    }
}
```

中间行的 16 个前导空格用边界符表示如下，竖线不属于程序输出：

```
|                GIG INC.|
```

这里把 strlen 的结果显式转换为 int，是为了与 WIDTH 和 spaces 使用同一种教学类型。该写法以名字长度能够由 int 表示为前提；本例的固定字符串显然满足。若名字长度超过 WIDTH，减法会得到负数，居中排版的前提已经失效，应先处理名字过宽的情况，而不是继续套用公式。

函数定义中的 ch 和 num 称为**形式参数**，简称**形参**。调用时提供的 SPACE 和 spaces 称为**实际参数**，简称**实参**。形参属于被调函数，用来接收本次调用的输入；实参属于调用表达式，可以是常量、变量或更复杂的表达式。

第二个实参也可以不先存入变量，直接写成表达式：

```
show_n_char(SPACE, (WIDTH - (int)strlen(NAME)) / 2);
```

调用前会先算出该表达式的值，再用结果初始化形参 num。两种写法的计算相同；把中间结果命名为 spaces 则更方便阅读和检查。

每个形参都要写自己的类型。void show_n(int x, y, z) 不是正确的原型写法，应写成：

```
void show_n(int x, int y, int z);
```

原型中可以省略形参名，只保留类型，例如 void show_n_char(char, int);。定义中需要实际使用形参，因此通常写出名字。

6.2.2　实参复制给形参

调用 show_n_char(SPACE, 12) 时，字符值和整数值分别用于初始化本次调用的 ch 与 num。C 语言的参数传递采用**传值**：被调函数得到的是实参表达式的值，不是调用者变量本身的别名。

因此，若实参是变量 spaces，被调函数修改自己的 num，并不会改写 spaces。若实参是表达式，也只把算出的结果交给函数。原型规定形参类型后，实参会按相应类型转换，规则与赋值转换相似。例如，show_n_char('*', 3.9) 中的第二个实参是 double，初始化 int 形参 num 时向 0 截断为 3，因此打印 3 个星号。浮点值的整数部分超出目标整数类型范围时，不能使用这条截断规则推导结果。

多个实参会被求值，但 C11 通常不规定各实参表达式按从左到右还是从右到左求值。不要写 show_n_char(i++, i) 来猜测哪一个实参先得到结果：这个调用既自增 i 又读取 i，两个动作之间没有规定的先后关系。把这类操作拆成前序语句，才能明确旧值和新值。

6.2.3 每次调用都有独立的局部对象

show_n_char 中的 cnt 只在该函数体内有效。main 也可以定义自己的 cnt，两个名字指代不同对象。名字在程序文本中的可见范围称为**作用域**。形参和普通局部变量具有**自动存储期**：每次进入所在函数或代码块时，本次调用使用自己的对象；离开后，这些对象不再可用。

下面的程序用 maximum 找出两个整数中的较大值。main 的对象是 a,b,c，maximum 的对象是 x,y,z。

```
#include <stdio.h>

int maximum(int x, int y);

int main(void) {
    int a = 5, b = 7, c;

    c = maximum(a, b);
    printf("a=%d, b=%d, c=%d\n", a, b, c);
    return 0;
}

int maximum(int x, int y) {
    int z;
    z = x > y ? x : y;
    return z;
}
```

输出为 a=5, b=7, c=7。调用前，main:c 尚未赋值，表中记为?，此时不能读取它。

```
before call          during maximum(a, b)      after return
main:a   = 5         main:a   = 5              main:a   = 5
main:b   = 7         main:b   = 7              main:b   = 7
main:c   = ?         maximum:x = 5             main:c   = 7
                     maximum:y = 7
                     maximum:z = 7
```

调用时，x 和 y 分别由 5 和 7 初始化，z 保存比较结果。返回后，本次 maximum 调用的形参和自动局部对象的生命期结束；main 的 a,b 仍保持原值，c 得到返回值 7。

一次普通调用可以按六个阶段理解：计算实参表达式；开始本次调用；用实参值初始化形参；执行函数体，并在各代码块内使用局部对象；返回时结束本次调用；主调函

数取得返回值（若有）并继续执行。嵌套调用会在外层调用尚未返回时开始内层调用，内外两层的形参和局部对象仍然各自独立。

6.3　返回值与返回路径

6.3.1　返回两个整数中的较小值

下面把较大值例子改为较小值，并把函数命名为 `imin`。`mini` 是 `imin` 的局部变量，`return mini;` 把它的值作为本次函数调用的结果。

```
#include <stdio.h>

int imin(int n, int m);

int main(void) {
    int ev1, ev2;

    while (scanf("%d%d", &ev1, &ev2) == 2) {
        printf("The smaller one of %d and %d is %d\n",
               ev1, ev2, imin(ev1, ev2));
    }
    return 0;
}

int imin(int n, int m) {
    int mini;
    if (n < m) {
        mini = n;
    } else {
        mini = m;
    }
    return mini;
}
```

输入 `5 7 -3 8 q` 时，前两行输出为：

```
The smaller one of 5 and 7 is 5
The smaller one of -3 and 8 is -3
```

第三次输入尝试遇到 q，本次没有完成两个赋值，循环结束。循环条件明确比较是否成功赋值两项，因此不会把上一次留下的值当成新输入。

若主调函数已有变量 lesser，可以写：

```
lesser = imin(n, m);
```

这里传给 lesser 的是返回值，不是 imin 内部的变量名。不能先调用 imin(n,m);，再写 lesser = mini;：mini 的作用域限于 imin，main 并不知道这个名字指代什么。

返回值还能直接参与更大的表达式，例如：

```
ans = 2 * imin(a, a + b) + 25;
printf("%d\n", imin(-21 + ans, NUM));
```

这些片段以变量均已正确定义和赋值、结果不超出类型范围为前提。函数也不一定要把某个局部变量原样返回，可以用表达式直接算出结果：

```
int imin(int n, int m) {
    return n < m ? n : m;
}
```

这与前面的版本使用同一个条件运算符，少了局部变量 mini，返回规则没有改变。

6.3.2 返回类型决定最终交回的值

函数头中的类型是**返回类型**。return 后面的表达式会先按这个类型转换，再作为函数结果交回调用者。下面的函数用浮点除法算出 1.5625，但声明的返回类型是 int。

```c
#include <stdio.h>

int what_if(int n);

int main(void) {
    int res = what_if(64);
    printf("res = %d\n", res);
    return 0;
}

int what_if(int n) {
    double z = 100.0 / (double)n;
    return z;
}
```

输出为 `res = 1`。局部变量 `z` 保存 1.5625；执行 `return z;` 时，该值按 `int` 转换，向 0 截去小数部分，得到 1。转换发生在返回时，`res` 接收到的已经是整数 1，不存在调用者再次截断的步骤。浮点值的整数部分超出 `int` 的可表示范围时，转换行为没有定义，不能预设输出。

6.3.3　提前返回与返回路径

`return` 不仅交回值，也立即结束本次函数调用。下面的写法在 `n < m` 时直接返回 `n`，其余情况执行最后的 `return m`：

```c
int imin(int n, int m) {
    if (n < m) {
        return n;
    }
    return m;
}
```

第一条 `return` 执行后，本次调用已经结束，不会再执行函数体中的下一条语句。两条输入路径都有明确的返回结果，因此不需要再写 `else`。

`void` 函数也可用 `return;` 提前结束。例如，一个打印倒数的函数可以在参数非正时直接返回，避免进入后续循环。除 `main` 外，非 `void` 函数要检查每条返回路径；如果执行到函数末尾仍未提供返回值，而调用者又使用这次调用的结果，程序行为没有定义。

6.4 递归：函数调用自身

6.4.1 进入下一层，再逐层返回

函数可以调用其他函数，也可以在满足一定条件时调用自身，这称为**递归**。下面的 up_and_down 在 n 小于 4 时调用下一层，并在递归调用前后各打印一次层次。

```
#include <stdio.h>

void up_and_down(int n);

int main(void) {
    up_and_down(1);
    return 0;
}

void up_and_down(int n) {
    printf("Level %d\n", n);
    if (n < 4) {
        up_and_down(n + 1);
    }
    printf("Level %d\n", n);
}
```

输出为：

```
Level 1
Level 2
Level 3
Level 4
Level 4
Level 3
Level 2
Level 1
```

前四行发生在递归调用之前，n 依次为 1、2、3、4。第 4 层满足 n < 4 为假，执行第二条输出语句后返回第 3 层。第 3 层从自己的递归调用之后继续，打印 Level 3 并返回第 2 层；同样的过程一直持续到第 1 层。每次返回都回到直接调用者，因此后四行的层次为 4、3、2、1。

四层调用各自有一个形参 `n`。当第 4 层正在执行时，前面三层都尚未返回，四个同名的 `n` 分别保存 1、2、3、4。某一层返回后，上一层的 `n` 仍保持原来的值，所以第 1 层最后打印的是 1。

6.4.2　选读：一帧调用与递归的机械展平

选读范围。本节用数据结构中常见的栈模型手工展开函数调用。后续正文不依赖本节的实现细节。

为便于记账，本节采用下面的简化模型。一次调用开始时，**主调函数**把两类信息放入栈中：传给被调函数的参数值，以及返回地址。返回地址表示被调函数结束后，主调函数应从哪个位置继续。随后，**被调函数**再加入自己的局部变量和准备交回的返回值。被调函数结束后，控制按返回地址回到主调函数；主调函数取走返回值，并清除这次调用占用的一整段栈内容。以上完整过程称为函数调用的一**帧**（frame）。

```
ONE FRAME: logical contents, not a byte layout
+--------------------------------------+
| caller part                          |
|   argument values                    |
|   return address                     |
+--------------------------------------+
| callee part                          |
|   local variables                    |
|   result value                       |
+--------------------------------------+
```

图中只列出一帧要记录的四类内容，不表示它们在真实内存中的字节顺序。新的帧压入栈顶，返回后由主调函数按约定清除；栈顶对应当前正在执行的调用。

以 `c = maximum(a,b)` 为例。设 `M0` 表示 `main` 中完成赋值的位置。调用过程可以记成：

```
PUSH a frame for maximum(5, 7)


TOP
+--------------------------------------+
| maximum frame                        |
| caller part: args x=5, y=7; ret=M0   |
| callee part: local z=7; result=7     |
+--------------------------------------+
| main frame: a=5, b=7, c=?            |
+--------------------------------------+
BOTTOM
```

主调函数 main 先记录参数值和返回地址 M0；被调函数 maximum 再加入局部变量 z 和返回值 7。返回时，控制转到 M0，main 取走 7，并清除 maximum 这一帧：

```
RETURN to M0, main takes result 7 and clears the frame

TOP
+--------------------------------------+
| main frame: a=5, b=7, c=7            |
+--------------------------------------+
BOTTOM
```

这个过程把两种边界画在一起：参数值随新帧进入被调函数，返回值离开被调函数；main 的 a,b 不因为 maximum 有了自己的局部变量而改变。

递归可以按同样的记账规则机械展平。把前面的递归公式改写成带局部变量的等价形式：

```
int rfact(int n) {
    if (n > 1) {
        int sub = rfact(n - 1);
        return n * sub;
    }
    return 1;
}
```

计算 rfact(3) 时，给三次调用编号。F1 调用 rfact(3)，F2 调用 rfact(2)，F3 调用 rfact(1)。设 R1 是 F1 中等待给 sub 赋值的位置，R2 是 F2 中相应的位置。最深处的栈如下：

```
TOP
+--------------------------------------+
| F3 frame                             |
| caller part: arg n=1; ret=R2         |
| callee part: no local sub; result=1  |
+--------------------------------------+
| F2 frame                             |
| caller part: arg n=2; ret=R1         |
| callee part: local sub pending       |
|              result 2 * sub pending  |
+--------------------------------------+
| F1 frame                             |
| caller part: arg n=3; ret=M0         |
| callee part: local sub pending       |
|              result 3 * sub pending  |
+--------------------------------------+
| main frame: answer pending           |
+--------------------------------------+
BOTTOM
```

压入过程把递归展开为 F1,F2,F3 三帧。每一帧都带有调用者准备的参数和返回地址，也带有被调函数自己的局部变量或返回值。返回时按相反次序收回：

```
1. F3 returns 1 to R2
   F2 clears the F3 frame; sub=1, result=2*1=2

2. F2 returns 2 to R1
   F1 clears the F2 frame; sub=2, result=3*2=6

3. F1 returns 6 to M0
   main clears the F1 frame; answer=6
```

这就是递归的机械展平：先按调用方向逐帧压入，再按返回方向逐帧清除。每一层只能回到自己的直接调用者，不能越过中间帧直接返回 main。若递归没有终止路径，压入会一直发生而没有相应的清除，调用栈最终会被耗尽。

真实的调用约定还会规定寄存器保存、栈对齐和返回值寄存器；由谁调整栈指针也随约定而变化。本节使用“主调函数记录参数与返回地址、被调函数加入局部变量和返回值、主调函数清帧”的模型，是为了机械展开调用过程。分析 C 程序的语义时，仍以参数按值初始化、局部对象和 return 的规则为准。

本节选读结束。

6.4.3 终止条件与语句次序

递归必须存在一条不再调用自身的路径，这条路径常称为**基例**或**终止条件**。本例的终止判断是 `n < 4` 为假；在这条从 1 开始的调用链中，第 4 层进入基例。还需要确认每层的实参会朝基例前进：这里每次用 `n+1` 调用，所以能够从 1 到达 4。若参数朝相反方向变化，或根本没有基例，调用链就不会按预期结束，最终可能耗尽实现可用的资源。

位于递归调用之前的语句，按照进入各层的顺序执行；位于递归调用之后的语句，按照返回各层的顺序执行。可以用缩进表示同一时刻尚未完成的调用：

```
up_and_down(1): print Level 1
    up_and_down(2): print Level 2
        up_and_down(3): print Level 3
            up_and_down(4): print Level 4
            up_and_down(4): print Level 4, return
        up_and_down(3): print Level 3, return
    up_and_down(2): print Level 2, return
up_and_down(1): print Level 1, return
```

这种“后执行的语句按相反层次出现”的性质可以用来反向输出。但递归并不是循环的等价替换语法：选择哪种写法，还要看问题结构、可读性和资源消耗。

6.4.4 阶乘：循环与递归的对照

非负整数 `n` 的阶乘记作 $n!$，规定 $0! = 1$，并在 $n > 0$ 时满足

$$n! = n \times (n-1)!.$$

这个等式可以直接写成递归：`n>1` 时返回 `n * rfact(n-1)`，否则返回 1。循环版本则从 1 开始，依次乘上当前的 `n` 并递减。

```c
#include <stdio.h>

long fact(int n);
long rfact(int n);

int main(void) {
    for (int n = 0; n <= 12; n++) {
        printf("%2d! = %12ld, recursive = %12ld\n",
               n, fact(n), rfact(n));
    }
    return 0;
}

long fact(int n) {
    long ans = 1;
    for (; n > 1; n--) {
        ans *= n;
    }
    return ans;
}

long rfact(int n) {
    if (n > 1) {
        return n * rfact(n - 1);
    } else {
        return 1;
    }
}
```

例如，计算 `rfact(4)` 的过程为：

```
rfact(4) = 4 * rfact(3)
         = 4 * (3 * rfact(2))
         = 4 * (3 * (2 * rfact(1)))
         = 4 * (3 * (2 * 1))
         = 24
```

递归调用发生在乘法完成之前。只有 `rfact(1)` 返回 1 以后，上一层的 `2 * 1` 才能

算出 2，再逐层得到 6 和 24。两个函数的使用前提为 $0 \le n \le 12$。即使 `long` 为 32 位，$12! = 479001600$ 也在可表示范围内；更大的输入需要先核对整个计算过程的类型范围。

把递归条件误写成 `while (n > 1)` 不能实现上面的公式。循环会在同一层调用里反复执行递归语句，而且 `n` 没有在该层递减；每次调用还未完成，就又发起下一次调用。这里应使用 `if`，让每层只递归一次，然后沿返回路径完成乘法。

对于只需逐项累乘的阶乘，循环通常更直接。递归版本需要保留多个尚未完成的调用；循环版本则用一个累计变量保存当前结果。递归的优势不在于所有问题都比循环快，而在于某些问题本身就按更小的同类子问题组织。

6.4.5　斐波那契数列：重复子问题

把前两项规定为 1，后面的每一项等于前两项之和，就得到本例采用的斐波那契数列：1、1、2、3、5、8、……。下面的递归定义直接对应这个规则，使用前提为 `n >= 1`。

```
#include <stdio.h>

long fib(int n);

int main(void) {
    for (int n = 1; n <= 10; n++) {
        printf("fib(%d) = %ld\n", n, fib(n));
    }
    return 0;
}

long fib(int n) {
    if (n > 2) {
        return fib(n - 1) + fib(n - 2);
    } else {
        return 1;
    }
}
```

计算 `fib(4)` 时，调用结构如下，每个函数名节点表示一次完整调用：

```
                fib(4)
              /        \
          fib(3)       fib(2)
         /     \          |
     fib(2)   fib(1)      1
       |        |
       1        1
```

这棵树上共有 5 次调用。沿 `fib(4)`、`fib(3)`、`fib(2)` 进入基例时，同一时刻最多有 3 层调用尚未返回。继续列出几项，可以看到“总调用次数”和“同一时刻最多未返回层数”的区别：

n	1	2	3	4	5	6
总调用次数	1	1	3	5	9	15
同一时刻最多未返回层数	1	1	2	3	4	5

总调用次数包括已经结束的重复调用，随下标增大呈指数级增长；最多未返回层数只随下标增大呈线性增长。前者影响运行时间，后者决定同一时刻需要保存多少层调用状态。

这个程序只演示小下标的递归关系。实际需要较大的斐波那契数时，可以从前两项迭代更新，避免重复子问题；同时还要检查结果类型能否表示该数。

6.4.6　十进制整数转成二进制

把一个非负整数反复除以 2，余数从低位到高位依次产生。以 11 为例：

$$11 = 2 \times 5 + 1, \quad 5 = 2 \times 2 + 1, \quad 2 = 2 \times 1 + 0, \quad 1 = 2 \times 0 + 1.$$

依次得到的余数是 1、1、0、1，打印二进制时却要先输出最后得到的最高位 1，再反向输出 0、1、1，得到 `1011`。递归调用后的语句正好提供这种反向次序。

```c
#include <stdio.h>

void to_binary(unsigned long n);

int main(void) {
    unsigned long number;

    if (scanf("%lu", &number) != 1) {
        return 1;
    }
    to_binary(number);
    putchar('\n');
    return 0;
}

void to_binary(unsigned long n) {
    int r = n % 2;
    if (n >= 2) {
        to_binary(n / 2);
    }
    putchar('0' + r);
}
```

计算 `to_binary(11)` 时，本层先保存余数 1，再调用 `to_binary(5)`；后续调用继续处理 2 和 1。最深层先打印最高位 1，返回后依次打印 0、1、1。输入 0 时不会递归，直接打印一个 `0`，所以不必另写一个特殊输出分支。

函数名直接调用自身称为**直接递归**。若 `subB` 调用 `subC`，`subC` 又调用 `subB`，形成间接回到自身的调用链，则称为**间接递归**。两者都需要明确的终止路径，也都要逐层返回。

6.5 声明、头文件与多文件编译

6.5.1 头文件提供接口信息

编译器分析一个函数调用时，需要知道返回类型和参数类型。函数定义在调用之前可以提供这些信息；定义位于后面或另一个源文件时，则需要先看到声明。标准库也采用同样的接口思路：`stdio.h` 提供标准输入输出函数的声明及相关宏和类型，`math.h` 提供数学函数的接口，例如：

```
double sqrt(double);
```

这条声明说明 sqrt 接收一个 double 并返回 double，但没有给出函数体。实现由标准库提供，编译和链接过程会把调用与实现连接起来。

下面的程序读入一个非负双精度浮点数并求平方根。输入失败或数值为负时，程序返回 1，不把不满足函数定义的输入交给 sqrt。

```
#include <math.h>
#include <stdio.h>

int main(void) {
    double x;

    if (scanf("%lf", &x) != 1 || x < 0.0) {
        return 1;
    }
    printf("sqrt(%g) = %f\n", x, sqrt(x));
    return 0;
}
```

输入 2 时，常见二进制浮点环境中输出 sqrt(2) = 1.414214。在某些编译环境中，数学库还需要显式参与链接，例如命令末尾添加-lm；这是构建方式的要求，不改变函数原型。

6.5.2　把声明和定义放进不同文件

程序增大后，可以把一个函数的接口放入头文件，把定义放入单独的源文件。下面三个文件组成一个完整程序。文件名只是本例的项目内名称，不是 C 关键字。

imin.h：

```
#ifndef IMIN_H
#define IMIN_H

int imin(int n, int m);

#endif
```

imin.c：

```c
#include "imin.h"

int imin(int n, int m) {
    return n < m ? n : m;
}
```

main.c:

```c
#include <stdio.h>
#include "imin.h"

int main(void) {
    printf("%d\n", imin(5, 7));
    return 0;
}
```

尖括号形式 `#include <stdio.h>` 用于实现指定的标准头文件搜索位置；双引号形式 `#include "imin.h"` 通常先查找当前源文件相关的位置，再按实现规定继续搜索。自定义头文件使用双引号，可以让两个源文件共享同一个原型。`imin.c` 也包含自己的头文件，使编译器能够核对声明与定义是否一致。

头文件通常放函数声明、类型声明和必要的宏定义，函数定义放在 `.c` 源文件中。`imin.h` 中的 `#ifndef`、`#define` 和 `#endif` 组成**包含保护**。第一次包含时，`IMIN_H` 尚未定义，预处理器保留声明并定义这个宏；再次包含同一个头文件时，条件不成立，就跳过直到 `#endif` 的内容。

6.5.3 编译单元与链接

一个源文件经过预处理后，连同所包含头文件的内容形成一个**编译单元**。编译器分别检查各个编译单元，生成保存编译结果和链接信息的目标文件；**链接器**再把目标文件和所需库组合成可执行程序，并解决跨文件的函数引用。

使用 GCC 时，可以一步完成本例的编译和链接：

```
gcc -std=c11 -Wall -Wextra -pedantic main.c imin.c -o imin_app
```

也可以分步执行：

```
gcc -std=c11 -c main.c -o main.o
gcc -std=c11 -c imin.c -o imin.o
gcc main.o imin.o -o imin_app
```

前两步分别编译，最后一步链接。若只在 main.c 中写声明而不提供任何 imin 定义，编译调用本身可能通过，链接时却会报告缺少定义。集成开发环境中的“项目”通常也是把多个源文件加入同一次构建，再由工具完成这些步骤。

6.6　伪随机数：可重复的序列与变化的种子

6.6.1　rand 的返回值与范围

stdlib.h 声明的 rand 返回一个 int，范围是 0 到 RAND_MAX，两个端点都包含在内。RAND_MAX 是由实现定义的宏，C11 保证它至少为 32767。通常的 rand 产生的是**伪随机数**：序列由确定算法和内部状态产生，并非不可预测的真随机源。

下面用固定种子初始化，再打印 10 个 0 到 99 之间的整数。

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    srand(10u);
    for (int i = 0; i < 10; i++) {
        printf("%d\n", rand() % 100);
    }
    return 0;
}
```

表达式 rand() % 100 的余数范围为 0 到 99。同一实现中，相同的种子和调用次序会重现相同序列；不同实现可以采用不同算法，因此具体数字可能不同。检查这个程序时，先核对行数和数值范围，再在当前实现中用固定种子复现序列。

6.6.2　用 srand 设置起点

srand 的原型为：

```
void srand(unsigned int seed);
```

它设置后续 rand 序列的起点。若希望每次运行得到不同序列，常见做法是用当前时间作种子：

```
#include <stdlib.h>
#include <time.h>

srand((unsigned int)time(NULL));
```

time(NULL) 返回当前日历时间，类型为 time_t；失败时可能返回 (time_t)-1。time_t 的具体类型和时间表示由实现决定。许多 Unix 环境用从 1970 年 1 月 1 日 0 时 0 分 0 秒起经过的秒数表示时间。把返回值转换为 unsigned int 可以生成普通测试种子；需要不可预测种子的场合应采用专门的随机源。

通常在一次程序运行开始时播种一次，然后连续调用 rand。若在每次取数前都用同一个值播种，就会不断把序列重置到同一起点；若时间值在快速循环中没有变化，也可能反复得到相同的首项。

余数映射适合演示范围。若所有可能的 rand 结果不能均匀分配到各个余数，简单取余就会带来分布偏差；例如 RAND_MAX 为 32767 时，共有 32768 个可能结果，不能平均分给 100 个余数。密码、密钥等安全用途也不使用 rand。

6.7 inline 是一项优化建议

函数调用有控制转移和参数准备等成本，短小函数被频繁调用时，编译器可能把函数体直接展开到调用处，省去一部分调用开销。C11 的**函数说明符** inline 用于表达这种内联展开的建议：

```
#include <stdio.h>

static inline int square(int x) {
    return x * x;
}

int main(void) {
    printf("%d %d\n", square(5), square(-3));
    return 0;
}
```

这里的 square 只在当前源文件中使用，因此写成 static inline：static 限定函数名在当前源文件内使用，inline 表达内联展开建议。输出为 25 9。

是否实际展开由编译器根据调用成本、代码尺寸和优化策略决定；普通函数即使没

有写 inline，也可能被展开。函数含有循环或递归时，同样由编译器判断是否适合展开。评估实际性能还需要查看生成代码并进行测量。

与宏相比，内联函数仍具有函数接口：参数有类型检查，实参按函数调用规则只求值一次。带参数的宏只是文本替换，若宏体多次使用某个参数，带有副作用的实参可能被多次求值。需要明确的参数类型和单次求值时，函数接口更容易检查；是否再添加 inline 则取决于实际优化需求。

6.8　指针：让函数找到调用者的对象

6.8.1　传值交换只修改形参

在 main 中直接交换两个变量，需要先保存其中一个旧值。若写：

```
x = y;
y = x;
```

第一句已经把 x 改成原来的 y，第二句读到的仍是这个新值，最后两个变量都等于原来的 y。正确做法是用 temp 保存旧值：

```
temp = x;
x = y;
y = temp;
```

把这三句原样放进一个接收两个整数的函数，却仍然不能交换调用者的变量。下面的 swap1 只交换自己的形参。

```c
#include <stdio.h>

void swap1(int x, int y);

int main(void) {
    int a = 3, b = 5;

    printf("before: a=%d, b=%d\n", a, b);
    swap1(a, b);
    printf("after:  a=%d, b=%d\n", a, b);
    return 0;
}

void swap1(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
}
```

输出中的两行都是 a=3, b=5。调用时，swap1:x 得到 3，swap1:y 得到 5；函数内部先把 temp 设为 3，再把两个形参改成 5 和 3。返回后这些形参不再可用，main:a 和 main:b 从未被写入。函数体“确实执行了交换”和“调用者的对象发生了交换”是两个不同命题。

要让被调函数修改调用者的对象，函数需要知道这个对象的位置。第二讲使用 scanf("%d", &dollar) 时已经见过这种需求：&dollar 提供变量的地址，输入函数据此把新值存到调用者的变量中。

6.8.2 地址与指针对象

对象在内存中的位置可以用**地址**表示。对变量 pooh 使用取地址运算符 &，表达式 &pooh 得到指向该对象的指针值。保存这个值的变量称为**指针对象**，简称指针：

```c
int pooh = 24;
int *ptr;
ptr = &pooh;
```

声明中的 int *ptr 表示 ptr 可以保存指向 int 对象的指针值。这里的星号属于

声明语法；在表达式里，星号还可能是乘法或解引用运算符，要结合位置区分。类似地，& 在表达式 a & b 中也可以是按位与，只有在 &pooh 这种一元用法中才表示取地址。

指针本身也是对象，有自己的存储空间和值；它所保存的地址指向另一个对象。int * 与 unsigned long 是不同的类型。指针的大小和表示由实现决定，与目标对象的大小不是同一回事；目标类型决定解引用后按什么类型读写。

可以用%p 输出指针值，参数应转换为 void *：

```
printf("%p\n", (void *)&pooh);
```

这里的 void * 表示不带具体目标类型的对象地址，%p 要求这种参数；它与函数返回类型中的 void 是不同用法。显示格式和具体地址由实现及本次运行决定，后文图用 @A 和 @B 作地址标签。

6.8.3　解引用：沿指针访问目标对象

若 ptr 保存 &pooh，表达式 *ptr 就表示 ptr 所指向的对象。这一操作称为**解引用**。它既可以出现在读取位置，也可以作为赋值目标。

```
#include <stdio.h>

int main(void) {
    int pooh = 24;
    int ans;
    int *ptr = &pooh;

    ans = *ptr;
    *ptr = 36;
    printf("pooh=%d, ans=%d\n", pooh, ans);
    printf("ptr=%p, &pooh=%p\n", (void *)ptr, (void *)&pooh);
    return 0;
}
```

第一行输出为 pooh=36, ans=24。执行 ans = *ptr; 时，目标对象仍保存 24，所以 ans 得到 24；随后 *ptr = 36; 写入的对象正是 pooh，因此 pooh 变为 36。第二行中的两个地址相同，文字形式不作固定要求。

在 ptr = &pooh 成立时，可以把下面几组说法对应起来：

表达式或说法	含义
ptr	指针对象当前的值，即 pooh 的地址。
&pooh	对 pooh 取地址，与当前 ptr 的值相同。
*ptr	ptr 指向的目标对象，在这里就是 pooh。
ptr 指向 pooh	可以通过 ptr 访问 pooh；不是把 pooh 复制进 ptr。

```
conceptual address          contents
       @A          ptr      |   @B   |
       @B          pooh     |   36   |

ptr contains @B, so *ptr designates the object pooh at @B.
```

图中 @A 和 @B 只是地址标签。这种指向关系没有复制 36；它表示 ptr 的值能用来找到地址 @B 处的对象。

只有指向有效对象的指针才能这样解引用。未初始化指针没有可靠目标，空指针不指向任何对象，对它们解引用都不是合法操作。函数若接收指针参数，也应把“必须指向怎样的有效对象”写成接口前提。

6.8.4 用指针完成交换

现在把两个形参改成指向 int 的指针。调用时传入 &a 和 &b，被调函数就能沿这两个指针访问 main 中的对象。

```c
#include <stdio.h>

void swap2(int *x, int *y);

int main(void) {
    int a = 3, b = 5;

    printf("before: a=%d, b=%d\n", a, b);
    swap2(&a, &b);
    printf("after:  a=%d, b=%d\n", a, b);
    return 0;
}

void swap2(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

这次第二行输出为 after: a=5, b=3。三条赋值的目标和来源都要沿指针辨认：

```
call: swap2(&a, &b)

main:a @A = 3 <--------- swap2:x = @A
main:b @B = 5 <--------- swap2:y = @B
                         swap2:temp

1. temp = *x   reads main:a, so temp becomes 3
2. *x = *y     writes main:b's value 5 into main:a
3. *y = temp   writes temp's value 3 into main:b
```

swap2 仍然采用传值调用：x 和 y 分别得到地址值 &a、&b 的副本。被调函数若只给自己的指针形参重新赋值，不会改变调用者变量；本例真正修改对象的是 *x 和 *y 的解引用写操作。

这个接口要求 x 和 y 都指向当前有效、可修改的 int 对象。若传入空指针或无法解引用的地址，三条交换语句就没有合法执行前提。

6.8.5　重新理解 scanf 的地址参数

比较下面两个函数调用：

```
function1(x);
function2(&x);
```

若原型分别是 `int function1(int num)` 和 `int function2(int *ptr)`，第一个调用把 `x` 的值复制给 `num`，第二个调用把 `x` 的地址复制给 `ptr`。后者允许函数在接口约定成立时修改 `x`。

`scanf("%d", &x)` 正是第二个思路的实际应用：`%d` 说明要把输入文本转换为 `int`，`&x` 说明结果应存到哪里。`printf("%d", x)` 只读取并显示数值，所以不需要地址。于是，前几讲输入与输出参数形式的不同，不再只是需要背诵的符号差异，而是“使用值”与“修改对象”两种需求的具体表现。

6.9 习题与解析

习题一：声明、定义与共享头文件

下面的声明想表示“函数不接收参数”，写法是否充分？若 `starbar` 的定义位于另一个源文件，调用它的源文件还需要什么？

```
void starbar();
```

解析：在 C11 的函数声明中，空圆括号没有提供参数个数和类型信息，不应把它当作明确的无参数原型。应写：

```
void starbar(void);
```

这条语句只是声明，不包含函数体。若定义在另一个源文件，可以把原型放入 `starbar.h`，让定义文件和调用文件都包含它；构建时还要把提供定义的源文件一起编译、链接。只有声明而没有定义，不能组成可调用的完整程序。

习题二：表达式作为实参

设 `spaces` 的值为 6，调用下面的函数会打印多少个星号？调用结束后 `spaces` 是否改变？

```
show_n_char('*', spaces + 2);
```

解析：实参表达式 `spaces + 2` 先算出 8，再用 8 初始化本次调用的形参 `num`，因此打印 8 个星号。传递的是表达式的值，不是表达式本身；即使函数修改 `num`，也不会改写调用者的 `spaces`，所以其值仍为 6。

习题三：局部结果怎样回到主调函数

在 maximum 例子中，为什么 main 不能直接用 z 得到较大值？若把调用写成 c = maximum(a,b);，调用后 a,b,c 各是什么？

```
int maximum(int x, int y) {
    int z;
    z = x > y ? x : y;
    return z;
}
```

解析：z 的作用域限于 maximum，main 中的名字解析不会找到这个局部变量。调用者使用的是函数调用表达式产生的返回值，而不是局部变量名。原来 a=5,b=7，调用中 x=5,y=7,z=7；返回后 c=7，而 a 和 b 仍为 5 和 7。

习题四：返回值经过几次转换

对前面的函数，分别求 what_if(64) 和 2 * imin(5,7) + 25 的值，并说明前者的 1.5625 在哪里发生变化。

```
int what_if(int n) {
    double z = 100.0 / (double)n;
    return z;
}
```

解析：what_if(64) 中，z 得到 1.5625；return z; 按函数声明的 int 返回类型转换，向 0 截去小数部分，所以调用结果为 1，不是在调用者的赋值语句中才截断。后一个表达式中，imin(5,7) 返回 5，因此 $2 \times 5 + 25 = 35$。返回值可以直接参与表达式，不必先存入 imin 内部的某个名字。

习题五：递归输出为什么先正序再逆序

写出 up_and_down(1) 八行输出中的层次编号。哪一层不再递归？若把第二条打印语句移到 if 之前的第一条打印语句旁边，输出次序会怎样？

```
printf("Level %d\n", n);
if (n < 4) {
    up_and_down(n + 1);
}
printf("Level %d\n", n);
```

解析：原代码的层次编号依次为 1、2、3、4、4、3、2、1。第 4 层不满足 `n < 4`，是这条调用链的基例。递归调用前的打印按进入层次出现，递归调用后的打印按返回层次出现。若把两次打印都放在递归调用之前，每层都会连续打印两次自己的编号，输出为 1、1、2、2、3、3、4、4，不再有返回时的逆序部分。

习题六：阶乘中为什么用 `if` 而不是 `while`

下面的递归函数想计算阶乘，存在什么问题？把它与正确的 `rfact(4)` 调用过程对照说明。

```
long rfact(int n) {
    long ans = 1;
    while (n > 1) {
        ans = n * rfact(n - 1);
    }
    return ans;
}
```

解析：在某一层中，`while` 条件使用固定的形参 `n`。递归返回后，该层的 `n` 并没有递减，因此循环会再次调用同一个 `rfact(n-1)`，无法按公式只完成一次乘法并返回。阶乘的每层只需要一次“计算更小阶乘并乘上 `n`”的操作，应使用 `if (n > 1) return n * rfact(n-1);`，基例返回 1。正确计算 `rfact(4)` 依次等待 3、2、1 阶乘，再返回 $1, 2, 6, 24$。

习题七：斐波那契调用次数与深度相同吗

设 `C(n)` 表示按本讲递归算法计算 `fib(n)` 所需的总调用次数。已知 `C(1)=C(2)=1`，求 `C(3)`、`C(4)` 和 `C(5)`。计算 `fib(5)` 时最多有几层调用同时活跃？

```
long fib(int n) {
    if (n > 2) {
        return fib(n - 1) + fib(n - 2);
    } else {
        return 1;
    }
}
```

解析：当 `n>2` 时，当前调用加上两个分支，所以 $C(n) = 1 + C(n-1) + C(n-2)$。因此 `C(3)=3`，`C(4)=5`，`C(5)=9`。沿每次都选择 `fib(n-1)` 的路径，可以得到 `fib(5)`、

`fib(4)`、`fib(3)`、`fib(2)` 这条最深链，共 4 层同时活跃，而不是 9 层。总调用次数描述整棵调用树，最大活跃深度描述其中一条最深路径。

习题八：余数为什么会反向打印

用本讲的递归算法把 13 转换为二进制。写出各层在进入下一层前保存的余数，以及最终输出。

```
void to_binary(unsigned long n) {
    int r = n % 2;
    if (n >= 2) {
        to_binary(n / 2);
    }
    putchar('0' + r);
}
```

解析：各层的 `n` 依次为 13、6、3、1，对应余数依次为 1、0、1、1。最深层的 `n=1` 不再递归，先打印自己的余数 1；返回后依次打印第 3、2、1 层保存的 1、0、1，所以输出 `1101`。余数产生的次序与打印次序相反，是因为每层把 `putchar` 放在递归调用之后。

习题九：随机种子与 `inline` 各保证什么

表达式 `rand() % 6 + 1` 的结果范围是什么？若在每次循环中都执行 `srand(10u)` 再调用一次 `rand()`，同一实现中可能出现什么现象？给函数加上 `inline` 是否保证一定展开并且一定更快？

```
for (int i = 0; i < 5; i++) {
    srand(10u);
    printf("%d\n", rand() % 6 + 1);
}
```

解析：`rand()` 的最小值为 0，取余结果范围为 0 到 5，再加 1 后范围为 1 到 6，包含端点。每轮都用 10 重新播种，会把序列重置到同一个起点，因此同一实现中五轮会打印相同的第一个映射值；应在循环前播种一次。`inline` 只是内联建议，不保证展开，也不保证速度提高，编译器仍会考虑成本、代码尺寸和优化策略。

习题十：两个交换函数差在哪里

分别以 a=3,b=5 调用 swap1(a,b) 和 swap2(&a,&b)。说明两个函数体中的三条交换语句分别修改谁，并写出调用后的结果。

```
void swap1(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
}

void swap2(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

解析：swap1 的 x,y 是 3 和 5 的副本，三条语句只交换这两个形参，返回后 a=3,b=5 不变。swap2 的 x,y 分别保存 &a,&b 的副本，但 *x 和 *y 指代调用者的 a,b。因此 temp=*x 读出 3，*x=*y 把 5 写入 a，*y=temp 把 3 写入 b，最后 a=5,b=3。两个版本都按值初始化形参，差别在于指针值使被调函数能够找到并修改调用者的对象。

6.10 检查一次函数调用

开头的信头程序把重复循环收进 show_n_char 后，调用者只需说明打印什么字符、打印多少次；交换程序则进一步要求传入地址，让函数找到调用者的对象。理解这两类调用，可以检查以下六项：

- **接口**：原型、返回类型、参数个数和参数类型匹配；无参数写成 void。
- **值与地址**：分清实参值、形参副本及是否修改调用者的对象。
- **对象**：分清每次调用的局部对象和解引用目标。
- **返回**：核对 return 的类型转换和各条路径的结果。
- **递归**：确认基例、参数推进和调用前后的语句次序。
- **构建**：声明可见、定义齐全，所需源文件和库参与链接。

函数把操作组织在清楚的接口之后。阅读一次调用时，应能说明接口两侧的数据、数据所属对象和返回路径。